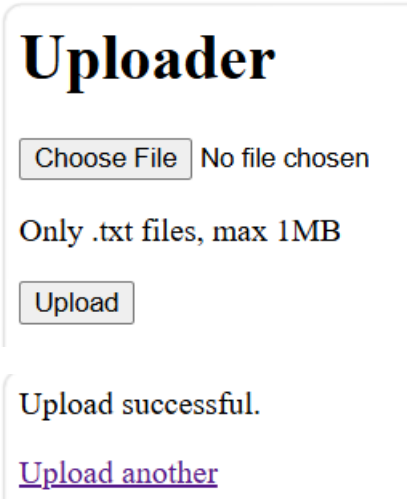
Submission Checklist

Screenshot of your local lab03/uploader-app/ and lab03/viewer-app/ folders showing app.js and package.json in each.

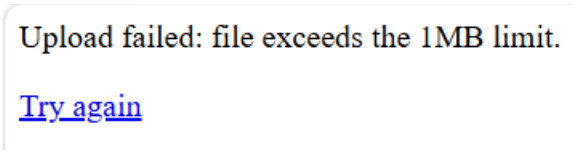
```
root@aa80cd592e15:/CloudClass/lab03/uploader-app# ls
app.js  package.json
```

```
root@aa80cd592e15:/CloudClass/lab03/viewer-app# ls
app.js  package.json
```

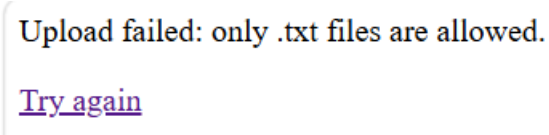
Screenshot of the Uploader page and a successful upload confirmation.



Screenshot of attempting to upload a .txt file larger than 1MB, showing the “file exceeds the 1MB limit” error.



Screenshot of attempting to upload a non- .txt file (e.g., a .jpg or .pdf), showing the “only .txt files are allowed” error.



Screenshot of the Viewer page displaying the uploaded file’s content.



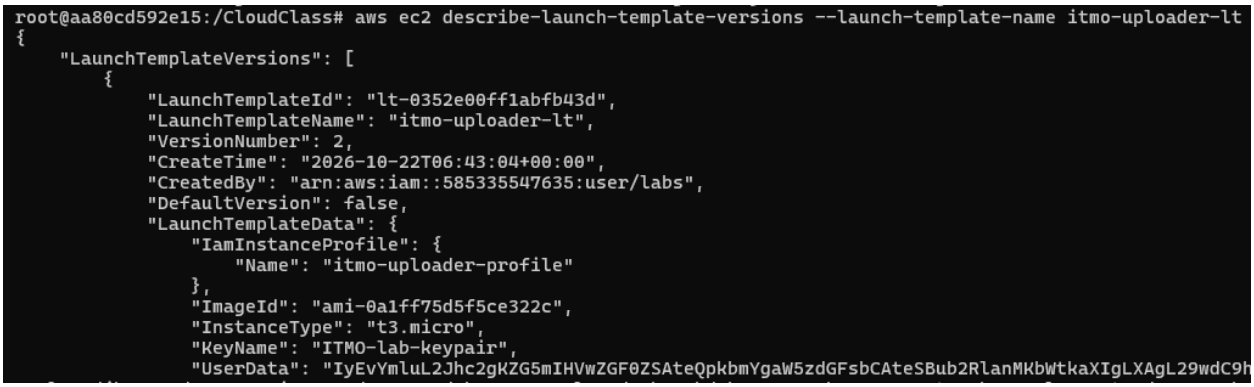
Screenshot of the Viewer page after uploading a second, different file and clicking Refresh, showing the updated content.

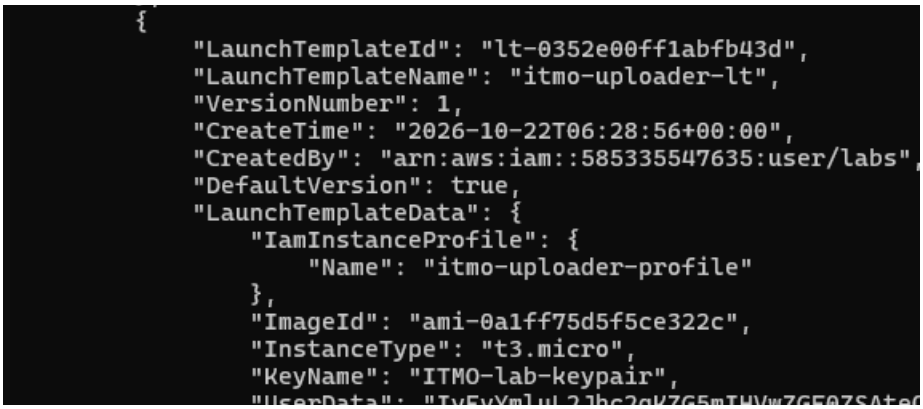


Screenshot of aws ec2 describe-launch-templates showing both templates.

```
root@aa80cd592e15:/CloudClass# aws ec2 describe-launch-templates --query "LaunchTemplates[*].LaunchTemplateName"
[
    "itmo-uploader-lt",
    "itmo-viewer-lt"
]
```

Screenshot of aws ec2 describe-launch-template-versions --launch-template-name <uploader-lt-name> after re- running the create script twice, showing more than one version.





aws iam get-role-policy output (or screenshot) for both roles, showing each has only its one intended action on the exact shared.txt key.

```
== Uploader role (PutObject on shared.txt only) ==
{
    "Role": {
        "Path": "/",
        "RoleName": "itmo-uploader-role",
        "RoleId": "AROAYQSFXU3ZYDRJXYQLN",
        "Arn": "arn:aws:iam::585335547635:role/itmo-uploader-role",
        "CreateDate": "2026-10-22T06:04:01+00:00",
        "AssumeRolePolicyDocument": {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Effect": "Allow",
                    "Principal": {
                        "Service": "ec2.amazonaws.com"
                    },
                    "Action": "sts:AssumeRole"
                }
            ]
        }
    }
}
{
    "InstanceProfile": {
        "Path": "/",
        "InstanceProfileName": "itmo-uploader-profile",
        "InstanceProfileId": "AIPAYQSFXU3ZVS3W7HCDV",
        "Arn": "arn:aws:iam::585335547635:instance-profile/itmo-uploader-profile",
        "CreateDate": "2026-10-22T06:04:07+00:00",
        "Roles": []
    }
}
== Viewer role (GetObject + ListBucket on shared.txt only) ==
{
    "Role": {
        "Path": "/",
        "RoleName": "itmo-viewer-role",
        "RoleId": "AROAYQSFXU3ZVNGGQOKGQ",
        "Arn": "arn:aws:iam::585335547635:role/itmo-viewer-role",
        "CreateDate": "2026-10-22T06:04:12+00:00",
        "AssumeRolePolicyDocument": {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Effect": "Allow",
                    "Principal": {
                        "Service": "ec2.amazonaws.com"
                    },
                    "Action": "sts:AssumeRole"
                }
            ]
        }
    }
}
{
    "InstanceProfile": {
        "Path": "/",
        "InstanceProfileName": "itmo-viewer-profile",
        "InstanceProfileId": "AIPAYQSFXU3ZTUQWOSCLH",
        "Arn": "arn:aws:iam::585335547635:instance-profile/itmo-viewer-profile",
        "CreateDate": "2026-10-22T06:04:18+00:00",
        "Roles": []
    }
}
```



Screenshot of create_app_stack.sh output showing both instances running with public IPs.

```
root@aa80cd592e15:/CloudClass# ./create_app_stack.sh
== S3 bucket ==
{
    "BucketArn": "arn:aws:s3:::itmo-gcm131-fileshare",
    "BucketRegion": "us-east-2",
    "AccessPointAlias": false
}
Bucket already exists, skipping.
== Uploader role (PutObject on shared.txt only) ==
Role exists, skipping.
Profile
exists, skipping.
== Viewer role (GetObject + ListBucket on shared.txt only) ==
Role exists, skipping.
Profile
exists, skipping.
Waiting 15s for IAM propagation...
== Building user-data from local lab03/ files ==
== Building launch template data ==
== Creating/updating launch templates ==
{
    "LaunchTemplates": [
        {
            "LaunchTemplateId": "lt-06bc7135afd33d3c7",
            "LaunchTemplateName": "itmo-uploader-lt",
            "CreateTime": "2026-10-20T05:50:45+00:00",
            "CreatedBy": "arn:aws:iam::585335547635:user/labs",
            "DefaultVersionNumber": 1,
            "LatestVersionNumber": 1,
            "Operator": {
                "Managed": false
            }
        }
    ]
}
Template itmo-uploader-lt exists - publishing a new version
{
    "LaunchTemplates": [
        {
            "LaunchTemplateId": "lt-0d18c5cd547918835",
            "LaunchTemplateName": "itmo-viewer-lt",
            "CreateTime": "2026-10-20T05:50:30+00:00",
            "CreatedBy": "arn:aws:iam::585335547635:user/labs",
            "DefaultVersionNumber": 1,
            "LatestVersionNumber": 1,
            "Operator": {
                "Managed": false
            }
        }
    ]
}
Template itmo-viewer-lt exists - publishing a new version
== Launching instances from the templates ==
Waiting for both instances to reach 'running'...
-----------------------------------
|        DescribeInstances        |
+--------------+------------------+
|  uploader-app|  18.117.244.233  |
|  viewer-app  |  3.141.5.121     |
+--------------+------------------+
```

```
-----------------------------------
|        DescribeInstances        |
+---------------+-----------------+
|  viewer-app   |  18.118.14.237  |
|  uploader-app |  77.114.68.211  |
+---------------+-----------------+
```

Screenshot of delete_app_stack.sh completing with no errors.



One paragraph explaining why the uploader and viewer use separate IAM roles instead of one shared role with both permissions.

The uploader and the viewer use separate IAM roles because of the principle of least privilege. If they were to use the same role and a bad actor were to get access to the new single unified role, they would be able to disrupt a lot more than if they were to get access to only one of the roles. For example, the unified role would give them both read and write capabilities and while it would be nice for them to have neither, we can mitigate it by splitting up the roles, so the bad actor is able to do as little harm as possible.

One or two sentences explaining why embedding code directly into user-data means the instance never needs a Git or S3 credential just to fetch its own code.

Normally code has to be fetched and you need a credential to do so however when the code is embedded inside user-data you bypass the need to fetch the code because its already embeded.